

VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs

Jingyi Xi^{*}
Chenghao Mo^{*}
SSAIL Lab, University of Illinois at
Urbana-Champaign
cmo8@illinois.edu

Benjamin Karsin
NVIDIA
bkarsin@nvidia.com

Artem Chirkin
NVIDIA
achirkin@nvidia.com

Mingqin Li
Microsoft
mingqli@microsoft.com

Minjia Zhang
SSAIL Lab, University of Illinois at
Urbana-Champaign
minjiaz@illinois.edu

Abstract

Vector search and database systems have become a keystone component in many AI applications. While many prior research has investigated how to accelerate the performance of generic vector search, emerging AI applications require running more sophisticated vector queries efficiently, such as vector search with attribute filters. Unfortunately, recent filtered-ANNS solutions are primarily designed for CPUs, with few exploration and limited performance of filtered-ANNS that take advantage of the massive parallelism offered by GPUs. In this paper, we present VecFlow, a novel high-performance vector data management system that achieves unprecedented high throughput and recall while obtaining low latency for filtered-ANNS on GPUs. We propose a novel label-centric indexing and search algorithm that significantly improves the selectivity of ANNS with filters. In addition to algorithmic level optimization, we provide architecture-aware optimizations for VecFlow’s functional modules, effectively supporting both small batch and large batch queries, and single-label and multi-label query processing. Experimental results on NVIDIA A100 GPU over several public available datasets validate that VecFlow achieves 5 million QPS for recall 90%, outperforming state-of-the-art CPU-based solutions such as Filtered-DiskANN by up to 135 times. Alternatively, VecFlow can easily extend its support to high recall 99% regime, whereas strong GPU-based baselines plateau at around 80% recall.

CCS Concepts

• Information systems → Data management systems; Semi-structured data.

^{*}Both authors contributed equally to this research.

[†]Work done while intern at UIUC.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

Keywords

Vector Database, Filtered ANNS, GPUs

ACM Reference Format:

Jingyi Xi, Chenghao Mo, Benjamin Karsin, Artem Chirkin, Mingqin Li, and Minjia Zhang. 2025. VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Dense vectors have become a key form of data representation in the era of AI. Deep learning models encode various entities, such as text [33, 36], images [31, 45], and code snippets [28], into dense continuous vectors, enabling a growing number of vector-based online applications, including image search [26], web search [22, 60], question and answering [58], ad-hoc retrieval [11, 20, 38], mobile search [2], and product search [53]. More recently, vector search has also become an integral component for building generative AI systems, such as retrieval-augmented generation for large language model (LLM)-based applications [30, 34]. To provide an interactive user experience at scale, these applications require high-performance deployment to achieve high throughput and recall with low query latency. As a result, there have been numerous studies on optimizing vector search performance by developing compute and memory-efficient approximate nearest neighbor search (ANNS) algorithms [4, 6, 14–16, 25, 27, 35, 39, 40].

Despite many advancements, prior work often focuses on optimizing *unconstrained top-K retrieval*, where the algorithm searches a vector dataset for the K closest vectors based purely on their geometric distance from the query. However, real-world and emerging applications often require finding Top-K results under certain *filtering* or *label* constraints. For example, Google Multisearch [18] allows users to search images with additional text hints, advertising engine apply filters to display region-relevant ads, and enterprise search places access control to displayable documents based on users’ permission levels.

One straightforward approach to enable ANNS with filters is to combine ANNS with *post-processing*, where one first runs a standard vector search over the entire dataset to obtain a list of candidates and then applies a filter operator on the given label to select the top-K

elements with the given filter. This approach allows one to leverage existing high-performance solutions developed for unconstrained top-K ANNS. However, it is very challenging to predict exactly how many returned candidates will pass the filter operation. Therefore, the number of final candidates that match the query's labels can be much less than K, or in the worst-case scenario, none at all.

In practice, post-processing methods perform poorly. As such, academia and industry companies such as Milvus [37], Weaviate [56], and Pinecone [44] that offer ANNS-as-a-service have explored alternative methods to support ANNS with filters. For example, Weaviate [56] builds an inverted index to search a query in an "approved list" of points that contain the query's labels. Alternatively, Filtered-DiskANN [17] proposes building a graph index with enhanced navigability for filtered searches and achieves state-of-the-art results with tens of thousands of query-per-second (QPS) at >0.9 recall. One gap in this approach is that Filtered-DiskANN [17] assume that a query only has one label while, in practice, queries can contain multiple labels, involving *OR* and *AND* operations. This is an important use case, so new filtered ANNS approaches should consider multiple query labels in their design.

ANNS index construction and search are computationally expensive operations, so modern GPU architectures offer an opportunity to significantly improve performance. Researchers have developed several GPU-based vector search algorithms for unconstrained top-K ANNS [41, 52, 62]. The state-of-the-art GPU-based vector search system from NVidia, CAGRA [41], builds GPU-friendly graph indices to accelerate the vector search on GPUs, offering over an order of magnitude higher QPS than CPU-based solutions. Despite achieving high ANNS speed on GPUs, few studies have explored high-performance design and implementations optimized for GPU-based filtered ANNS, due to several challenges. First, the hardware architecture of GPUs is very different from CPU. While the GPU has a much higher parallelism and memory bandwidth than CPU, one has to redesign the parallelism strategy for an algorithm to map it to the GPU architecture to extract sufficient parallelism to fully utilize the hardware resources. Second, directly applying filter-aware partitioning algorithms leads to huge memory redundancy, yet GPU global memory is a scarce resource. Therefore, it is important to optimize the memory efficiency of GPU-based filtered-ANNS. Third, existing GPU-based ANNS algorithms primary focus on maximizing throughput when the batch size is large. However, in online serving scenarios, queries are often coming in through small batch sizes, making it difficult to achieve high GPU utilization while maintaining a low query latency.

In this paper, we investigate the following research question: *How to build a high-performance system for filtered ANNS that can effectively leverage GPU hardware?* To address the aforementioned challenges, we present VecFlow, a high-performance vector filtered-search system for GPUs. Different from recent graph-based solutions [17, 41], VecFlow adopts a label-centric IVF design to improve the selectivity of ANNS with filters. In addition, VecFlow divides the IVF posting lists into distinct groups, each with its own characteristics of label distribution. We then design a dual-structured index and search algorithms for each group, maximizing the GPU utilization to achieve fast and accurate filtered ANNS (§ 4.1). Fig. 1 shows the overview of VecFlow's design. Moreover, we develop high-performance GPU kernels (§ 4.2) to achieve (1)

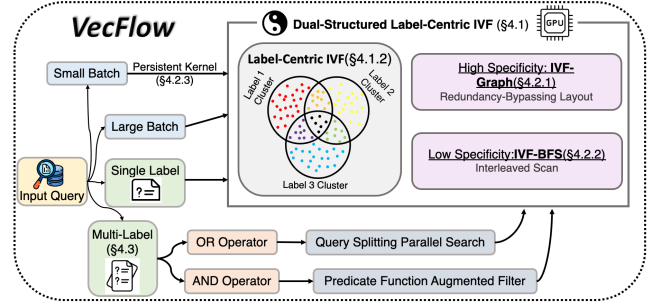


Figure 1: System overview of VecFlow.

redundancy-bypassing IVF-Graph (§ 4.2.1), (2) interleaved scan based IVF-BFS (§ 4.2.2), and (3) persistent kernels for streaming small batches (§ 4.2.3). Together, these optimizations enable unprecedented filtered-ANNS search performance. Finally, different from prior work that assumes each query only has one label, we introduce a high-performance algorithm to tackle queries with multiple labels with *OR* and *AND* conditions among labels (§ 4.3).

Our experiments show that VecFlow achieves significantly speedups over state-of-the-art filtered-ANNS solutions on both CPU and GPU over both real-world and semi-synthetic filtered datasets. In particular, VecFlow achieves 5 million QPS for recall 90%, outperforming state-of-the-art CPU-based solutions such as Filtered-DiskANN by up to 135 times. Alternatively, VecFlow can easily extend its support to reach the high recall regime ($>99\%$), whereas strong GPU-based baselines plateau at around 80% recall for these use cases.

2 Background

2.1 Filtered ANNS

As one of the core components for data management, there are numerous algorithms for ANNS with diverse index construction methods and a range of trade-offs concerning indexing time, search time, and recall [4, 6, 14–16, 25, 27, 35, 39, 40]. Several work have done an excellent survey and comparison of ANNS algorithms [3, 32, 42, 55]. However, most existing research addresses the unconstrained ANNS problem from the perspective of improving search efficiency [14, 35], reducing the memory and compute costs [25, 46], enabling fresh updates to the index [50], and unifying SQL and vector search [61]. With the increasingly central role of ANNS in dense retrievers, new capabilities are required for ANNS, including *search with filters* (filtered-ANNS).

There has been several recent works on filtered-ANNS, such as Analytic DB-V [57], NHQ [54], IVF² [5], and Filtered-DiskANN [17]. Among these, IVF² is an IVF-based (inverted file index) method that wins the NeurIPS'23 Big-ANN Competition Filter Track[8]. Filtered-DiskANN achieves excellent results by building filtered search on top of Vamana graph-based ANNS solution DiskANN [24]. In particular, Filtered-DiskANN builds separate proximity graph for data points associated to each label and stitches multiple small graphs together to eliminate redundant edges. By leveraging the geometric relation between points and incorporating label information during index construction, Filtered-DiskANN achieves much better search efficiency than prior filtered-ANNS methods based on IVF/LSH indices. Unlike these existing filtered-ANNS solutions, ours is the first to explore achieving optimal filtered-ANN performance on GPUs.

Furthermore, different from prior work that only considers simple filters, such as exact matches with one filter in Filtered-DiskANN, we address more complex filtering conditions, including efficient search with the disjunction (OR) and conjunction (AND) of multiple filters, which have been less explored in previous research.

2.2 GPU-based Vector Search

The high computational throughput of modern GPU architectures has made it a key resource to solve computationally difficult problems [21]. In recent years, increasingly computationally difficult operations, such as ANNS, have become commonplace in database systems, necessitating hardware acceleration [13, 47]. Early work to efficiently accelerate ANNS using GPUs focused on developing fast k-selection algorithms based on product quantization [10, 26]. Subsequently, several studies proposed to build graph-based search algorithm on GPUs [19, 52, 59, 62, 63]. For example, GGNN builds hierarchical kNN graphs on GPUs [19]. BANG [52] employs a heterogeneous CPU-GPU vector search solution, using compressed data for distance computations on GPUs while maintaining the graph and actual data points on the CPU. Although these methods achieve promising speedups, they primarily focus on adapting or optimizing GPU resource utilization over existing CPU-designed graphs, without fully leveraging the GPU’s capabilities. More recently, Ootomo et al. [41] proposed CAGRA, an enhanced GPU algorithm for solving k-ANNS. CAGRA designs hardware-friendly proximity graph structures to fully leverage the compute and memory bandwidth of modern GPU, outperforming both well-known CPU-based ANNS such as HNSW [35] and state-of-the-art GPU-based ANNS, such as GGNN [19] and GANNS [59], achieving the state-of-the-art results for k-ANNS. Different from those work, VecFlow focuses on solving the filtered-ANNS problem on GPUs. In addition, although VecFlow leverages CAGRA graphs, it is fundamentally different. First, CAGRA can only perform k-ANNS without filters. Although CAGRA can be combined with post-processing to support filtered-ANNS, the performance is quite sub-optimal. In contrast, VecFlow’s novel index and search algorithms are specifically designed to enable significantly higher QPS and recall on filtered-ANNS. Second, while CAGRA achieves high throughput with very large batch sizes, VecFlow introduces a persistent kernel-based method to achieve filtered-ANNS high throughput in streaming scenarios with small incoming query batch sizes.

3 Problem Formulation

DEFINITION 1 (FILTERED NEAREST NEIGHBOR SEARCH (FILTERED-NNS)). Given a finite data point set X of N points in a vector space $\mathcal{R}^D$, each data point (a.k.a. vector) $x \in X$ has an associated set of labels $L_x \subseteq \mathcal{L}$, where $\mathcal{L}$ is a finite set of labels. The goal of filtered-NNS is to answer a given query point $q \in \mathcal{R}^D$ with labels $L_q \subseteq \mathcal{L}$ by finding the closest point $x \in X$ with L_p , i.e., points in X that have the label L_p associated with them.

Note that the query point q may or may not be in the point set X , and the above definition generalizes naturally to *Top-K Filtered-NNS*, i.e., finding $A_{topk} = \{x\}$ top-K nearest points to q in X . Given that N is often quite large and the service level objective (SLO) is stringent, we define an approximated form of the above problem.

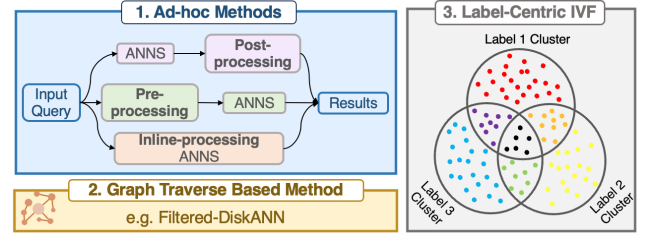


Figure 2: Three common ways for Filtered-ANNS.

DEFINITION 2 (FILTERED APPROXIMATED NEAREST NEIGHBOR SEARCH (FILTERED-ANNS)). The goal of ϵ -filtered-NNS is to design an algorithm that answers a given query point $q \in \mathcal{R}^D$ with labels $L_q \subseteq \mathcal{L}$ by maximizing the recall of finding the top-k closest points $A_{topk} \in X$ with L_p while minimizing the search time spent to return A_{topk} .

Assume the ground truth top-K nearest neighbors of q is GT_{topK} , the recall is defined as follows:

$$\text{Recall} = \frac{|A_{topK} \cap GT_{topK}|}{|GT_{topK}|} = \frac{|A_{topK} \cap GT_{topK}|}{K} \quad (1)$$

4 Design and Implementation of VecFlow

This section describes the design and implementation of VecFlow.

4.1 Dual-Structured Label-Centric IVF

We start by considering two popular approaches for ANNS with filters, namely the graph traversal-based approach such as Filtered-DiskANN [17], and the clustering-based method employed in Milvus [37] and Weaviate [56], which relies on inverted file indexing (IVF) [29]. Filtered-DiskANN proposes to exploit both geometric relationship between data points and the labels that each point has to construct a navigable graph index and perform graph traversal with inline-filtering, which achieves excellent performance in Filtered-ANNS and outperforms filtered-ANNS methods based on IVF/LSH indices on CPU.

Despite its promising results, Filtered-DiskANN uses a single graph index for all points, meaning that multiple queries with different labels must use the same method to search this single graph index. When the number of data points with associated labels is high, the single index approach performs similarly to unfiltered search. However, we observe that the label distribution in many datasets exhibits a *long tail* distribution, with a few frequent labels and many rare ones, as shown in Figure 3. Challenges arise when data points with certain rare labels fall into the long tail, as most distance computations in the index search process become unnecessary, negatively impacting query performance. In addition, it is difficult to ignore vectors that do not pass the filter during graph traversal, as this can break the connectivity that the graph-based index relies on to achieve high recall. When the graph is large and the frequency of data points corresponding to a given label is very low, the search performance of Filtered-DiskANN becomes problematically low. As we later show in the evaluation, FilteredVamana achieves a maximum recall of only 50%, while StitchedVamana reaches at most just 70% on the YFCC dataset. Both methods struggle to reach high recall because YFCC has many rare labels, making it challenging to predict how many top-K returned candidates will include those rare

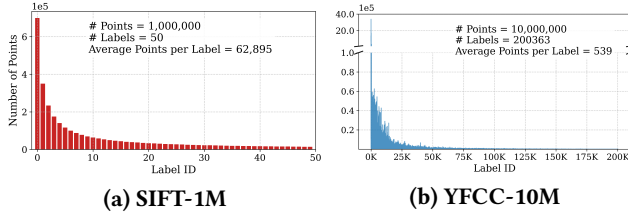


Figure 3: Label specificity distribution in different datasets.

labels. Consequently, the number of final candidates that match the query’s labels is less than K . Moreover, a large graph degree is required to maintain search quality (96 for FilteredVamana and 64 for StitchedVamana, as reported in [17]). This further impacts search efficiency, as a higher graph degree results in slower search performance. This motivates us to consider algorithms that leverage the high computational throughput of GPUs to achieve high recall even on these cases with rare labels, which may be prohibitively costly on CPU systems.

Different from graph-based methods, IVF indexing is another fundamental technique in ANNS that partitions vectors into clusters to reduce the search space [9, 25]. In traditional IVF methods, vectors are clustered based on their distance to the cluster centroids (typically using k -means clustering), creating “inverted lists” where the cluster centroid becomes the keyword, i.e., the descriptor of the cluster, and the posting list comprises all vectors in the corresponding cluster. During a search, the system first identifies the most promising clusters by comparing the query vector with centroids, then performs exact searches only within those selected posting lists. In contrast to traditional IVF, one can construct an inverted index where the keyword is the label, and the posting list contains vectors that share the same label [5, 44]. This label-centric IVF knows exactly which posting list to search for each query based on its labels, eliminating the cluster selection overhead. This approach works effectively like setting $n_probes = 1$ in traditional IVF search, but without the accuracy trade-off, as it guarantees searching the exact cluster containing the matching vectors. Prior studies show that IVF-based filtered-ANNS solutions can also achieve competitive performance compared to graph-based solutions such as Filtered-DiskANN [49].

To address the issue, we introduce a *dual-structured label-centric IVF* that leverages the high computational throughput and parallelism of GPUs for high-performance filtered-ANNS. VecFlow adopts a label-centric IVF design to improve the selectivity of ANNS with filters, because a query only needs to search the posting list that has the query’s label. In addition, we define label specificity, a metric for quantifying the density of a given label in a dataset, and we divide the IVF posting lists into distinct groups, each with its own characteristics of label distribution. VecFlow employs a dual-structured index and search algorithms to achieve fast and accurate filtered ANNS. For each group, we design high-performance GPU kernel optimizations to maximize GPU resource utilization. We first describe this algorithm and then introduce several optimizations to map the algorithm to GPUs.

4.1.1 Specificity Definition We start with the definition of *specificity*, a property that measures the fraction of data points in X that

have the label $l \in \mathcal{L}$ associated with them. More specifically:

$$\text{specificity}(l) = \frac{\text{number of data points with label } l}{\text{total number of data points } N}$$

This definition is similar to prior work [24, 43]. Different from prior work, we further classify a label as:

- High-specificity (HS): $\text{specificity}(l) \geq T$
- Low-specificity (LS): $\text{specificity}(l) < T$

where T is a chosen specificity threshold. A data point x therefore can also be categorized according to its labels:

- High-specificity data: x has at least one high-specificity label
- Low-specificity data: x has at least one low-specificity label

Note that in scenarios where a data point can have multiple labels, a data point can have both high- and low-specificity labels simultaneously.

4.1.2 Decoupling High/Low-Specificity Labels with Label-Centric IVF and Dual-Structured Indices We construct an inverted index where the *keyword* is the label, and the posting list contains vectors that share the same label. Specifically, for each $l \in \mathcal{L}$ and a subset of data points $X_l = \{x | l \in L_x \text{ and } x \in X\}$, we build a posting list C_l over X_l . Once we build the Label-Centric IVF, we divide the posting lists into two groups: the high-specificity group (HS) and the low-specificity group (LS), and develop a dual-structured index construction and search algorithm to enable efficient filtered-ANNS for both HS and LS. For the HS partition, we build a separate GPU-friendly graph index G_l (e.g., CAGRA [41]) over an individual posting list C_l (IVF-Graph)

This allows VecFlow to avoid an exhaustive search within a posting list while taking advantage of the hardware-friendly GPU-based ANNS solution, achieving performance similar to unfiltered search. For the LS partition, we directly resort to brute force search (IVF-BFS).

The rationale for adopting this dual-structured index is that for LS data, each IVF list contains a relatively small number of points, making the construction and maintenance of a graph-based index unnecessarily complex and potentially counterproductive. For example, CAGRA’s search behavior is primarily governed by its internal top- k (*itopk*) parameter rather than the actual dataset size. This parameter determines both the number of iterations and the number of distance computations performed, regardless of cluster size. Our experimental analysis demonstrates this scaling inefficiency — when testing across different cluster sizes from 100 to 20,000 points with constant *itopk*(32), CAGRA’s throughput remains relatively flat (2.6M QPS for 100-point clusters vs. 2.3M QPS for 20,000-point clusters), as shown in Fig. 4(a).

Despite showing excellent performance, graph traversal incurs iterative neighbor expansion that leads to additional synchronization overhead. Fig. 4(b) demonstrates that for queries with low-specificity labels in the YFCC dataset, an average of 71.29% of the CAGRA search time (*itopk*=32) is allocated to actual distance computations, while the remaining time is distributed across graph traversal overhead, including initialization (8.04%), parent node selection (5.41%), and internal top- k maintenance (14.79%). Consequently, when a label corresponds to only a small number of points, it becomes more efficient to directly compare the query vector against all points in the IVF list, avoiding the overhead of initializing and traversing a graph.

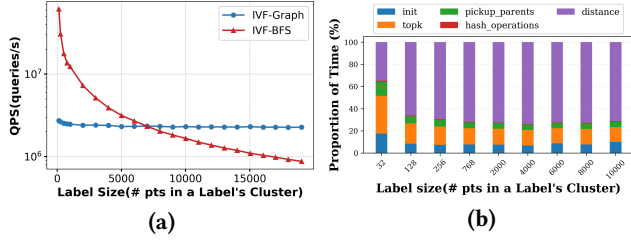


Figure 4: (a) Scaling efficiency of IVF-Graph and IVF-BFS varying the number of data points within a cluster. (b) Time decomposition of CAGRA search process.

For low-specificity data, BFS offers advantages over graph-based methods because BFS can be more easily parallelized to efficiently leverage modern GPUs, especially when the number of data points is relatively small. Essentially, the distance computation between a query and a set of vectors can be effectively formulated as a *batched matrix-vector multiplication (GeMV)* operation, which can be tiled across Streaming Multiprocessors (SMs) on a GPU. This approach helps to maximize the utilization of the GPU’s compute and memory bandwidth. GeMV is primarily a memory bandwidth bound operation and the peak memory bandwidth for a GPU (e.g., Nvidia A100) is often over 2 Terabytes per second (2TB/s), which is over 20x higher than modern CPU bandwidth (often <100GB/s). Thus, BFS on GPUs offers significant speedups over CPUs, on which BFS may be prohibitively slow. By employing BFS, execution time is optimized by focusing on the distance computation, eliminating graph traversal overhead entirely. This makes BFS a more efficient choice for low-specificity scenarios where the benefits of graph traversal are minimal.

During the search, whether a query searches the HS or LS partition is determined by the specificity threshold T . Given a query with label L , we route the query based on the size of its corresponding cluster $|C_L|$:

$$\text{method} = \begin{cases} \text{IVF-BFS} & \text{if } \frac{|C_L|}{N} < T \\ \text{IVF-Graph} & \text{otherwise} \end{cases} \quad (2)$$

Finding the optimal threshold T analytically is non-trivial as it depends on many architectural parameters and specificity distribution. However, it is not necessary in practice. T can be carefully chosen to represent the crossover point where the brute force approach becomes less efficient than graph traversal for a given number of data points. This is determined through offline profiling query performance across different cluster sizes and auto-tuning, e.g., by comparing the performance of HS and LS search to identify a threshold T that maximizes the overall query throughput.

4.2 Index and Search Strategies of VecFlow on GPUs

4.2.1 Bottom-Level IVF-Graph with Redundancy-Bypassing for HS

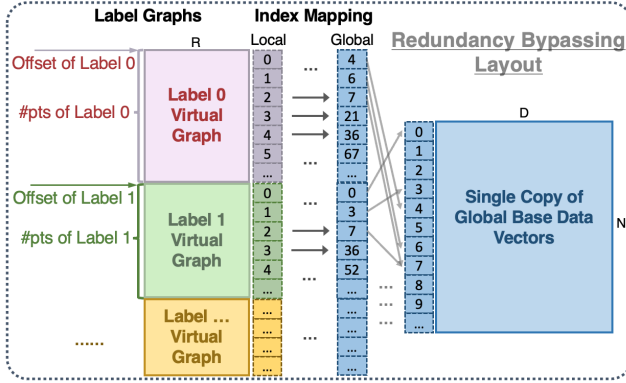
Indexing. While combining IVF with graphs significantly improves the search speed, building an individual graph for each posting list leads to one major challenge – the memory consumption becomes much higher compared to a single index for all data points. We observe that directly building IVF-Graph index for each posting list leads to a 10.8x increase in memory compared to a single

graph index over the entire YFCC dataset. Why does the memory consumption increase dramatically? We dive deep into how graph-based ANNS index are constructed for GPUs and find out that this is caused by memory redundancy in IVF-Graph. Taking CAGRA as an example, for a given set of data points, CAGRA builds a k-NN graph with fixed out-degree using NN-descent [12] on GPUs and performs rank-based reordering to improve the graph’s navigability. The final CAGRA index consists of two components: a graph structure consisting of the geometric relation, and an embedding table that stores all vector embeddings in a 2D $N \times D$ continuous memory buffer. Note that for most datasets and configurations, the vector embeddings dominate memory consumption compared to the graph topology structure, e.g., a data vector size of 192 in the YFCC dataset versus a neighbor list size of 16. This would not cause an issue if the partitioning of IVF lists are non-overlapping. However, in practice, multiple labels may be associated with a data point, resulting in a single point being replicated into multiple inverted lists if that data point contains multiple labels.

To address the redundancy issue, we propose an approach called IVF-Graph with Redundancy-Bypassing. For each posting list in the label-centric IVF, we build a *local virtual* graph index, where each virtual index only contains the graph structure information expressed via local vertex IDs, and a *global* embedding table that contain all vector embeddings. In addition, we maintain a **local-global** mapping table that translates these local vertex IDs (within each posting list) to their corresponding global indices in the global embedding vector table. This mapping mechanism allows us to maintain only a single copy of the global embedding table while having multiple virtual graph indices to share the same embedding table.

Current graph-based ANNS algorithms typically use a single graph structure represented by adjacency lists, where each row uniquely corresponds to the neighbors of a specific data point. However, our method introduces multiple virtual indices to enable filtered search. If these virtual indices were stored separately, processing queries with different labels would require frequent index switching, hindering the parallel performance of methods like CAGRA for large batches. On the other hand, storing multiple graphs together in a single index is not supported by existing ANNS libraries. To address the issue, we design a readily integrable and GPU-efficient index structure to implement our approach. In particular, we compact all virtual graphs into a single, continuous memory space, organized and ordered by their label IDs. This compacted structure not only avoids the overhead of switching between indices but also represents multiple virtual graphs as a unified structure. As a result, it seamlessly integrates into any graph-based ANNS algorithm by replacing the original single-graph structure, enabling filtered searches without requiring significant modifications to the index structure. Furthermore, since our combined index is compatible with single-graph structures, any optimization techniques designed for single indices can be directly applied to our unified graph.

The layout of our compact graph index is illustrated in Fig. 5. To make this structure functional, we maintain metadata for each label to identify its specific position and structure within the compacted neighbor list array. This metadata includes: (1) *Label Size*: The number of points in the label’s cluster $|C_L|$ (e.g., # pts of Label 1



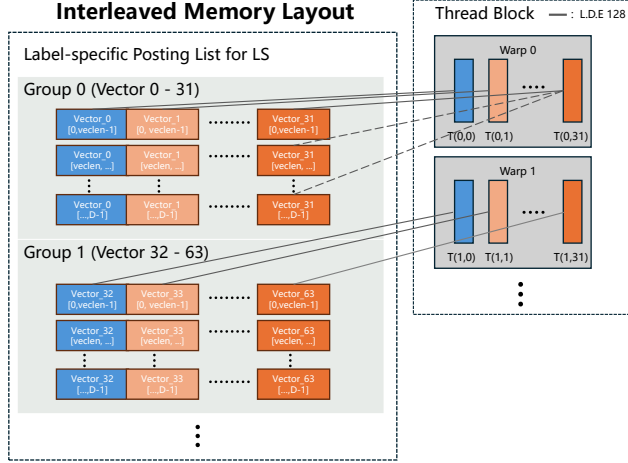


Figure 6: Interleaved memory layout for VecFlow’s IVF-BFS over LS partition.

block, the query vector is first loaded into shared memory for fast access by all threads. The block then processes vectors in its assigned posting list through coordinated warp execution. Each warp processes one group of 32 interleaved vectors from the memory layout described in Figure 6. Within a warp, each thread computes the distance between one vector from the current group and the shared query vector. Another important step during the brute force search is to filter out the top- k candidates. The kernel fuses an optimized block-select- k operation into the distance computation to identify nearest neighbors efficiently. This approach outperforms cuVS’s brute force search implementations by avoiding extra distance computations through label-specific IVF posting lists and achieving better memory bandwidth utilization with optimized interleaved memory layout. This hierarchical organization naturally aligns with the structure of small label-based clusters: blocks can independently process different query-label pairs, while warps provide efficient parallel distance computation and sorting within each cluster. Overall, this ensures complete coverage for exhaustive search without the overhead of graph traversal.

We note that, for low-specificity data points, our approach introduces additional memory overhead compared to using base vectors directly. This overhead arises because vectors are stored within each label’s IVF list in a continuous, interleaved layout to optimize GPU memory bandwidth utilization. Since a single data point may be associated with multiple low-specificity labels, its vector needs to be stored in multiple IVF lists. The total memory overhead can be expressed as $N \times D \times F$, where F represents the average number of low-specificity labels per point. For example, in the YFCC dataset, about 5.95M points (59.5% of the dataset) have at least one low-specificity label (e.g., labels associated with fewer than 1,000 points), with these points appearing on average in 3.54 low-specificity IVF lists (21.1M total entries across low-specificity IVF lists divided by 5.95M points), leading to 3.54x memory overhead compared to the original vector storage for LS partition.

4.2.3 Persistent Kernel-based Search for Small Batch Queries Since all queries within a batch are independent, we use a single-kernel implementation for handling batched queries but use an individual

CUDA thread-block (single-CTA) within a kernel to handle large batch sizes, similar as how CAGRA handles large batch sizes. Overall, when the batch size is large (e.g., >100), each query is mapped to a single thread block, and multiple of these blocks are executed in parallel. This approach effectively harnesses the massive parallelism of GPUs, maximizing their compute potential. However, when the batch size is small (e.g., <100), the GPU resource can be left underutilized with the single-CTA implementation.

In practice, vector search services often must execute queries in small batches and maintain low latency. However, traditional GPU-based methods like CAGRA are optimized for large batches and suffer from high overhead when processing small batches due to repeated kernel launching overhead. One may wonder whether it is possible to hide the kernel launching overhead to some extent by launching a few kernels in parallel using streams, thereby overlapping the launch overhead with the execution of other kernels. However, this strategy also does not work very well in practice because of CPU thread synchronization overheads. Modern GPUs require hundreds of thread blocks running in parallel to achieve maximum throughput. At this scale, even a few CUDA host API calls per-thread can become a bottleneck due to shared CUDA context locking. To address this, we introduce a persistent kernel approach that maintains a continuously running kernel on the GPU to process queries as they arrive, which not only eliminates the repeated kernel launch overhead but also avoid synchronization at all costs by using atomic-based control structures.

Fig. 7 shows our persistent kernel design, which uses a job queue system with atomic ring buffers to manage incoming queries and a worker queue to track available GPU thread blocks. When a query arrives, it is assigned a job ID and mapped to an available worker. The persistent kernel continuously monitors for new work, with each thread block handling one query independently. This approach enables efficient parallel processing while maintaining high GPU utilization despite small batch sizes.

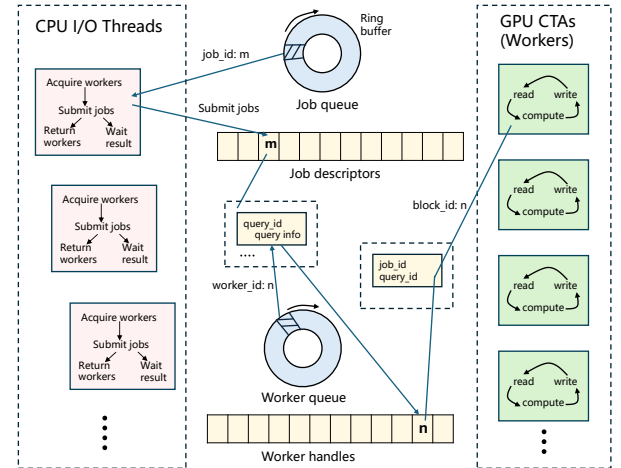


Figure 7: Persistent kernel design in VecFlow.

4.2.4 Memory Consumption. We express the memory consumption as the sum of bytes required to store both the HS and LS partition indices. Let F denote the average number of labels per point, with F_{HS} and F_{LS} representing the averages for the HS

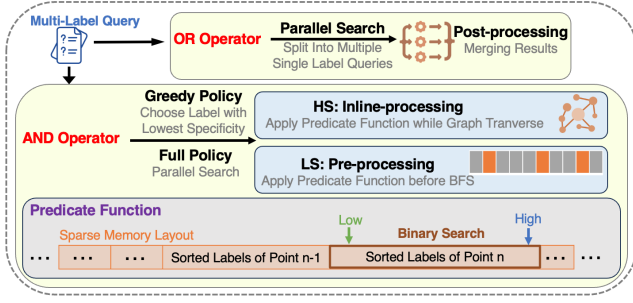


Figure 8: Multi-label query processing in VecFlow.

and LS partitions, respectively. Without applying the redundancy-bypassing algorithm, the memory consumption of the HS partition is $N \times F_{HS} \times (D+R) \times b$, where R is the length of the out-degree, and b is the number of bytes of data type. After applying the redundancy-bypassing optimization, VecFlow stores a single shared copy of the base data vectors while maintaining separate neighbor lists for each label-specific graph. This reduces memory usage significantly because D is often much larger than R . The memory consumption of the HS partition becomes $N \times (D + F_{HS} \times R) \times b$. Furthermore, the average cluster size per label is much smaller than N in practice. For example, in the YFCC dataset, this average label size reduces from 10M to 540. Consequently, we use a much smaller out-degree R' , which shows no performance degradation based on empirical observation. Thus, the final memory consumption for the HS partition is $N \times (D + F_{HS} \times R') \times b$. Instead, we store only the base data in the LS partition, resulting in a memory consumption of $N \times D \times F_{LS} \times b$. Thus the total memory consumption is $N \times (D + F_{HS} \times R') \times b + N \times D \times F_{LS} \times b$. Assume we have 100 million points, with the following parameters: $D = 128$, $R = 64$, $F = 3.17$, $F_{HS} = 3.0$, $F_{LS} = 0.17$, $b = 4$, $R' = 16$. Substituting the values, the memory consumption is 65.57 GB + 8.11 GB = 73.67 GB, which is slightly higher than a single index's memory consumption of 71.53 GB, calculated as $N \times (D+R) \times b$. For metadata, the total memory consumption for local-global index mapping is $N \times F \times b$, which is 1.18 GB. Regarding label metadata (*Label Size* and *Label Offset*, 2 integers for each label), since the number of labels is typically in the range of several thousands, its memory usage is negligible. For example, even in the YFCC dataset with 200K labels (which already represents a relatively large label set), the label metadata only accounts for 1.49 MB. This value is sufficiently small that it can be ignored in most cases.

4.3 Multi-Label Query Processing with Predicate Filtering and Early Stopping

Prior work often assumes queries have a single label. However, in practice, one query is often associated with multiple labels, which introduces challenges. Multi-label queries require different processing strategies based on whether the labels are combined with *OR* or *AND* operations. For *OR* operations, we need to find points that match any of the query labels, while *AND* operations require points to satisfy all query labels simultaneously. These distinct requirements lead us to develop specialized approaches for each operation type.

4.3.1 Handling OR Operations For multi-label query connected by *OR* operations, we can simply convert it into single-label search problem, treating each label as a separate search request and launch multiple standard single-label searches *in parallel*, using IVF-Graph search for high-specificity labels and IVF-BFS for low-specificity labels, followed by merging results to find the nearest neighbors that satisfy any of the query labels.

4.3.2 Handling AND Operations It becomes challenging to handle *AND* operations efficiently because a point must satisfy all query labels simultaneously. For example, when looking for points that satisfy both label A *AND* B, even if label A and B individually have many matching points, the set of points satisfying both constraints may be sparsely distributed in the single graph index. This forces the search to explore many irrelevant paths and points before finding valid results that satisfy all constraints. Our key insight to support efficient *AND* Operations is that it is *not necessary to check all paths* to identify candidates that satisfy A *AND* B. Instead, we develop a predicate function that efficiently verifies whether a point contains all specified labels and uses this predicate function to obtain a *selective early-stopping* search strategies for both HS and LS partitions to handle *AND* queries effectively.

Predicate function for efficient multi-label verification. To efficiently verify whether a data point contains all specified labels, we use a compact data structure based on three arrays: a global label array, a label offset array, and a label size array. The global label array stores the label of all data points sequentially, keeping each data point's labels contiguous and sorted. The label offset array marks where each point's labels begin in the global array, and the label size array records how many labels each point has. We choose this structure over a matrix representation $N \times |\mathcal{L}|_{max}$ (where $|\mathcal{L}|_{max}$ represents the max label size), because of highly variable label sizes. In the YFCC dataset, while the average is 10.8 labels per point, some points have up to 1,500 labels. A matrix approach would waste memory by allocating 1,500 columns for every point. Our array-based approach stores only the actual labels each point has, reducing memory fragmentation and enabling efficient GPU parallel processing through a compact, contiguous memory layout.

For each data point, we employ a *binary search* algorithm to retrieve its portion of labels from the global label array. For instance, if a data point's offset is 1000 and it has 15 labels, we search within positions 1000 to 1014 of the global array. The search adjusts two pointers iteratively based on comparisons with the target label. If found, it returns true and the label's position; otherwise, it returns false, indicating the point fails the *AND* predicate.

For queries with multiple labels, we optimize the search process by first sorting the query labels. It then searches from the smallest and largest query labels to establish the valid range that must contain all other query labels. For example, suppose a point has labels [1,3,5,7,9,11,13,15] and the algorithm needs to verify query labels [5,9,11]. It first searches for the smallest label 5, finding it at index 2. If label 5 is not found, it immediately returns false. Otherwise, it searches for the largest label 11, finding it at index 5. If label 11 is not found, it returns false. After finding both boundary labels, it establishes that all remaining query labels must exist within range [3,4]. For the middle label 9, it only needs to search

this smaller range. This approach maintains the worst-case complexity of $O(|\mathcal{L}_q| \log F)$, but typically performs better in practice as intermediate searches operate on a much smaller range. The function returns true only if all query labels are found.

IVF-Graph AND search for HS. For queries with multiple labels connected by *AND* operations in the high-specificity partition, we develop a *greedy search policy* that combines IVF list selection with inline-processing during graph traversal. The key idea is to minimize the search space while maintaining high accuracy through two main components. Our search space selection is straightforward: given a query with multiple labels connected by *AND* operations, the search selects the IVF list corresponding to the label with the *lowest* specificity. For example, if a query asks for points with labels A associated with 10,000 points AND B associated with 1,000 points AND C associated with 5,000 points, the search chooses label B's IVF list. This ensures we search within the smallest possible search space. For each iteration of graph traversal, we employ the inline-processing approach that integrates label verification directly into the search process. After computing distances at each graph expansion iteration, we apply our predicate function to check whether candidate points contain all the rest higher-specificity labels from the query. Points that fail this verification step are removed from consideration before the next iteration begins. This filtering strategy ensures that graph traversal expands only from points that satisfy all label requirements.

This combined approach provides performance benefits through improved selectivity. By starting from the IVF list of the lowest-specificity label and using higher-specificity labels as filters during traversal, we increase the relative selectivity to points that satisfy all label constraints. For example, when searching for points with labels A AND B where A is associated with 50,000 points and B with 5,000 points, searching in B's IVF list reduces the initial search space by 90% compared to searching in A's list. Meanwhile, since a point in the dataset has a higher probability of having label A, filtering for label A during graph traversal becomes more effective. This means when we verify if a point has label A during traversal, we have a better chance of finding valid neighbors, making the search more efficient in exploring the relevant regions of the graph.

If very high accuracy is desired, we also provide a full *parallel search policy* for handling AND queries in the HS partition. This approach converts AND queries into single-label search problems, similar to what we do for OR operators, but incorporates inline processing with our predicate function. Instead of starting from the IVF list with the lowest specificity, we launch parallel searches in each label's IVF list. Each search applies the predicate function during graph traversal to verify if points contain all other labels in the query. For instance, when searching for points with labels A AND B, two parallel searches are launched: one in label A's IVF list filtering for label B, and another in label B's IVF list filtering for label A. Each search is executed by a dedicated CTA. After all CTAs complete their searches, the final results are obtained by merging the top-K candidates from all parallel searches. This method delivers higher QPS, particularly when the recall target is high (e.g., >90%).

IVF-BFS AND search for LS. For queries with multiple labels connected by AND operations in the low-specificity partition, we select the IVF list corresponding to the label with the lowest specificity.

This choice is sufficient since BFS will traverse all points in the selected list. Before performing a brute-force search within the selected IVF list, we apply our predicate function to verify whether each point satisfies all query labels. This pre-processing strategy effectively eliminates unnecessary distance computations for points that do not meet all label constraints.

5 Evaluation

In this section, we experimentally evaluate VecFlow.

5.1 Evaluation Methodology

Datasets. We use public datasets from prior work to evaluate our system, as listed below:

- **Semi-synthetic dataset.** Following the approach used in Filtered-DiskANN [17], we employ Zipf's law to generate labels for the base dataset, effectively simulating the skewed distributions commonly observed in real-world applications. For the base dataset, we use **SIFT-1M**(D=128), generating a total of 50 labels with an average of 3.17 labels assigned per data point. Additionally, to assess the scalability of our algorithm, we use same method to generate 2,500 labels for the **DEEP-50M**(D=96) dataset with an average 5.88 labels per point.
- **YFCC.** The YFCC-10M dataset[51], utilized in the BigANN Filter competition, contains a total of 200,386 labels with an even more skewed distribution than labels generated from Zipf's law. On average, each label corresponds to only 540 points, while each data point is associated with an average of 10.8 labels.
- **WIKI-ANN.** Wiki-ANN [23] is a dataset introduced in [7], designed to evaluate AND predicates. Each query in the dataset is associated with two labels connected by an AND operator. The complete dataset comprises 35 million embeddings, each with 768 dimensions, where the embeddings alone exceed 100 GB in size, surpassing the memory capacity of a single GPU. To make it manageable, we use a 1-million-point subset provided by the dataset to assess multi-label search performance. This subset utilizes the 4,000 most frequent words as labels, with each data point having an average of 22.5 labels.

Metrics. We evaluate Filtered-ANNS search efficiency using recall and queries processed per second (QPS). The recall measures the fraction of the top-K query results retrieved by the Filtered-ANNS that match exact nearest neighbors with query labels(measured by brute force search). In practice, we usually choose $K = 10$ and take QPS at 90% recall as an important metric. In addition, we also measure average latency (time to run a single batch) in latency mode (single-batch mode), and memory footprint consumption of our algorithm.

Baselines. We compare with the following baseline methods:

- Filtered-DiskANN [17]: We include the two algorithms from Filtered-DiskANN: FilteredVamana and StchedVamana. We use the parameters listed in the Filtered-DiskANN paper to build the index and search.
- IVF² [5]: The best-performing solution from the NeurIPS'23 Competition BigANN Filter track that uses CPU for filtered-ANNS. It is tailored for the YFCC dataset.

- CAGRA + Post-processing (CAGRA-Post) [41]: We include a strong baseline by extending CAGRA with post-processing. We build a CAGRA graph for the entire dataset, query as usual, and employ post-processing to select only those results returned by the index that match the query filter.
- CAGRA + Inline-processing (CAGRA-Inline) [41]: This offers a stronger baseline by extending the state-of-the-art GPU-based ANNS solution CAGRA with on-the-fly filtering, i.e., the graph traversal skips points in the graph that do not match the filters of the query.

Testbeds Experiments are conducted on nodes with NVIDIA A100 GPU (40GB) and AMD EPYC 7763 CPU (2.45 GHz, 128 cores, 256GB memory) and NVIDIA Grace Hopper GH200 Superchip (96GB).

5.2 Main Result

Our first experiment demonstrates the search performance across different datasets. We compare VecFlow with the state-of-the-art CPU-based Filtered-DiskANN, IVF², and our GPU-based baselines CAGRA-Inline and CAGRA-Post. For Filtered-DiskANN, we build graphs with $L = 90$ and $R = 96$ for FilteredVamana, $R_{small} = 32$, $L = 100$ and $R_{stitched} = 64$ for StitchedVamana; For IVF² we use parameters suggested in [48]. For CAGRA-Inline and CAGRA-Post we build a single CAGRA graph with $R = 32$ as suggested in [41]. For VecFlow, we set the specificity threshold $T = 2000$, and we build the CAGRA graphs with $R = 16$ for the HS group.

Our results, as shown in Fig. 9 that follows the style of [5, 17, 41], highlight VecFlow’s significant advantage in QPS-vs-recall across different datasets. We achieve these plots by varying search parameters for each algorithm to improve recall and reduce throughput: search width (L_s from [17]) for FilteredVamana and StitchedVamana, and *itopk* for the CAGRA-based approaches and VecFlow. Overall, it is worth noting that VecFlow achieves million-scale QPS at 90% recall ($K=10$), which is one to two orders of magnitude higher than both CPU and GPU baselines.

On the semi-synthetic SIFT-1M dataset, FilteredVamana achieves a maximum recall of 60% even with a large L_s value. This suggests that traditional proximity graph with inline-filtering cannot effectively capture label-specific information within the graph’s connections. StitchedVamana performs better than FilteredVamana, achieving 37K QPS at 90% recall and capable of reaching 100% recall. This is because the stitched graph retains some label-related connections, which helps improve filtered graph search. CAGRA with post-processing and CAGRA with inline-processing both outperform StitchedVamana, thanks to CAGRA’s highly optimized search capability on GPUs. In comparison to those existing methods, VecFlow achieves an impressive 5M QPS at 90% recall, which is 135 times higher than StitchedVamana. VecFlow is able to achieve high performance because the dataset has only 50 labels, all of which are high specificity (the least specific label cluster contains 14,000 data points). This causes each query to only search a small IVF list, resulting in significantly reduced distance computation. In addition, VecFlow leverages CAGRA’s high performance within each IVF, achieving high throughput and high recall by leveraging GPU-friendly graph traversal. This use case illustrates that VecFlow performs very well for datasets without extreme label distributions.

Despite the more extreme filter labels for the YFCC-10M dataset, we observe similar trends with the performance of various methods.

FilteredVamana struggles with lower recall levels even with large L_s , highlighting its limitations in capturing label-specific connections on a larger dataset. StitchedVamana also begins to struggle to achieve high recall, getting less than 70% even with very large L_s . This is because the YFCC dataset contains over 200K labels, many of which exhibit low specificity. For this dataset, IVF² performs the best among the CPU baselines, achieving 48K QPS at 90% recall. This is because it employs a label-centric IVF method similar to ours, which is particularly well-suited for datasets like YFCC that have a large number of labels with low specificity. CAGRA + Post-processing and CAGRA + Inline-processing outperform StitchedVamana again, leveraging GPU efficiency for rapid search, but are unable to achieve more than 80% recall. Finally, VecFlow continues to outperform all other methods, achieving a remarkable QPS of 2.6M at 90% recall, demonstrating its scalability and effectiveness on a real dataset as large as YFCC-10M and as complex as 200K labels.

The WIKI-1M dataset is particularly challenging due to the multi-label search queries. FilteredVamana and StitchedVamana are not included in the results because they do not support multi-label filter searches. Both CAGRA Post-processing and Inline-processing methods achieve nearly 0 recall due to the extremely small intersection between two labels, making it unlikely to find points with the same label as the query. In contrast, VecFlow still delivers high performance, achieving 150K QPS at 90% recall. This is attributed to our highly efficient predicate function and optimized search policy for multi-label queries described in § 4.3. Finally, our evaluation on DEEP-50M, which is the largest dataset that can accommodate on a NVidia A100 40GB GPU, reveals that VecFlow achieves around 3M QPS at 90% recall, whereas other methods struggle to surpass 50% recall and many fail to even reach 20% recall, which is consistent with our previous analysis over SIFT-1M.

5.3 Analysis Results

Next we will justify each part of our design by conducting related performance evaluations. We also evaluate VecFlow’s effectiveness over other type of GPUs.

Label-centric IVF improves the selectivity of filtered-ANNS. To consider the impact of filtering on overall performance, we measure the performance of CAGRA search with the SIFT-1M dataset without any label filtering. Fig. 10 plots the performance of unfiltered CAGRA and VecFlow using the synthetic filters discussed previously. We see that, despite the overhead and complexity of applying filtering, VecFlow demonstrates excellent performance and is even 1.6x faster than CAGRA search without filters at 90% recall. This advantage arises because VecFlow employs a label-centric IVF structure, letting it only search the specific cluster containing the query’s label. This limits the search space of each query and reduces unnecessary computation.

Another observation is that reducing the degree from 32 to 16 in CAGRA unfiltered search results in a performance drop of approximately one-third at $> 90\%$ recall. Furthermore, in the high-recall region, the performance with degree 32 consistently outperforms degree 16. However, VecFlow exhibits almost no performance degradation when the degree is reduced from 32 to 16. This robustness is also attributed to the label-centric IVF design, which ensures

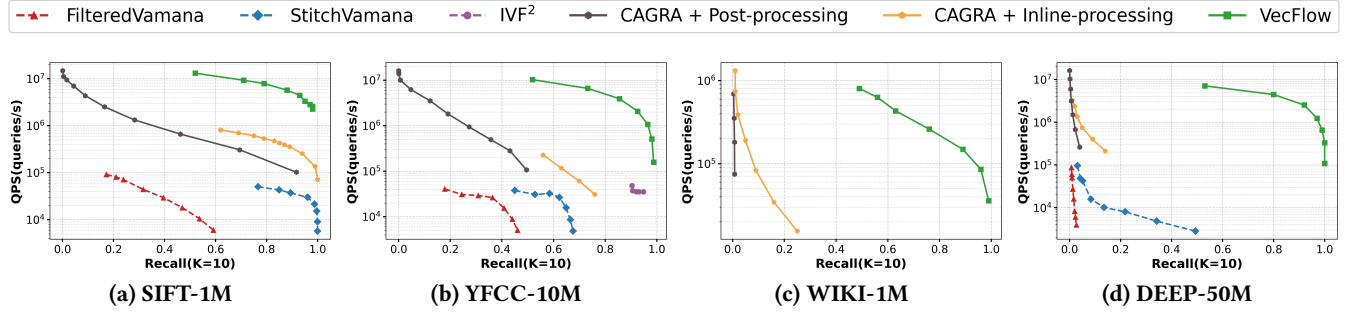


Figure 9: The QPS vs. recall comparison results on (a) SIFT, (b) YFCC, (c) WIKI and (d) DEEP respectively.

that each graph is responsible for a smaller, more focused subset of data. This not only enhances performance but also reduces memory requirements by allowing us to maintain a smaller degree in the graph structure.

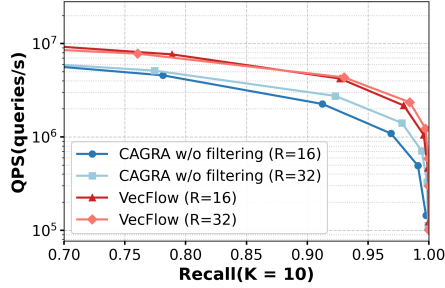


Figure 10: Comparison results between VecFlow with filters and CAGRA without filters.

How does redundancy-bypassing IVF-Graph bring benefits? In § 4.2.1 we introduce the IVF-Graph with redundancy-bypassing (RB) optimization. We evaluate its effectiveness over SIFT-1M and WIKI-1M datasets by comparing (i) single-index graph-based approach using CAGRA with out-degree of 64, (ii) IVF-Graph without RB, and (iii) IVF-Graph with RB. In addition, we also report IVF-Graph with reduced degree. Fig. 11 shows that without RB, the memory consumption of IVF-Graph increases dramatically, especially when each data point is associated with a large number of labels. In contrast, the RB optimization drastically reducing index size to 0.95 $\times$ and 1.38 $\times$ the single-index baseline for SIFT-1M and WIKI-1M, respectively. This is achieved through our local virtual graph plus local-global mapping that enables sharing of the embedding vectors across IVF lists, effectively eliminating redundancy.

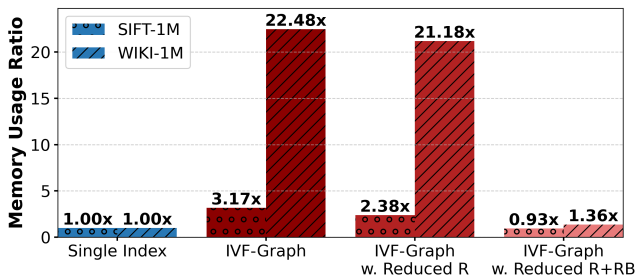


Figure 11: Memory efficiency of redundant bypassing.

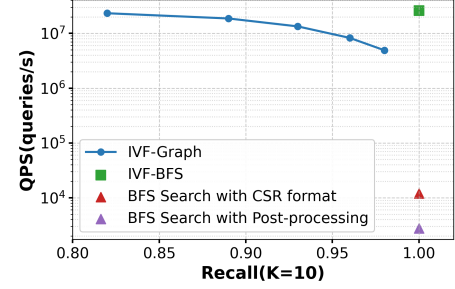


Figure 12: IVF-BFS performance analysis on YFCC dataset ($T = 2000$).

Is it necessary to perform IVF-BFS for search in LS? Fig. 12 presents the performance comparison between IVF-BFS and two baseline methods on the YFCC dataset for clusters with sizes less than 2000. The results show that IVF-BFS achieves 26M QPS, which is 2167 times and 9455 times faster compared to baseline BFS search with CSR format and BFS search with postprocessing, respectively. Additionally, it is 1.32 times faster than IVF-Graph at 90% recall. This performance advantage comes from our interleaved scan-based kernel optimization, which makes IVF-BFS suitable for accelerating search in LS.

How does VecFlow's multi-label search policy perform? Fig. 13 illustrates the comparison between the two search strategies we proposed for the AND query. Here, we discuss the advantages and disadvantages of both approaches. The results show that the parallel strategy outperforms the greedy policy, especially in the high-recall region. For instance, on the wiki dataset, the greedy policy achieves a maximum recall of only 94% even with the largest *itopk*. However, with the same *itopk* size, greedy policy searches only the cluster with the lowest specificity label, resulting in reduced computational costs while achieving faster search speeds. As shown in the figure, with the same *itopk*, the QPS of the greedy policy is significantly higher than that of the parallel strategy, with minimal loss in accuracy.

We also evaluate single-label and double-label queries separately on the YFCC dataset. As shown in Fig. 16, both types of queries can achieve high recall of $> 97\%$. Regarding QPS, single-label queries are 4 times faster than double-label queries at 90% recall, as the inline-processing slows down the search for double-label queries.

How does persistent kernel improve the performance of small batch queries? We test VecFlow on single-batch mode. For QPS, the persistent kernel achieves an average speedup of 5.68x and a maximum

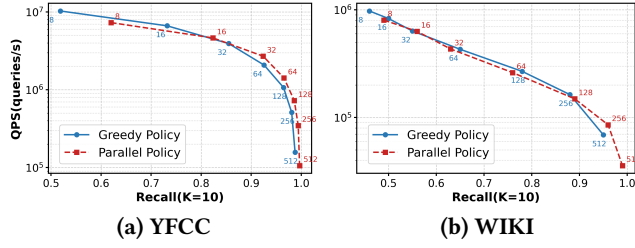


Figure 13: The comparison of multi-label search policy in VecFlow. The *itopk* size used for each point is labeled next to the corresponding data point.

speedup of 6.39x on the SIFT dataset than the baseline, as well as an average speedup of 6.72x and a maximum speedup of 7.08x on the YFCC dataset. For the average latency per query, the persistent kernel achieves an average speedup of 1.54x and a maximum speedup of 1.71x on the SIFT dataset, and an average speedup of 1.73x and a maximum speedup of 1.82x on the YFCC dataset, as shown in Figure 14. Persistent kernel-based search improves the QPS when batch size is small because it uses a single-kernel to process incoming queries, without launching additional kernels, largely reducing the kernel launching overhead. In addition, by continuously accepting new queries to process, it effectively increases the GPU utilization, achieving high QPS.

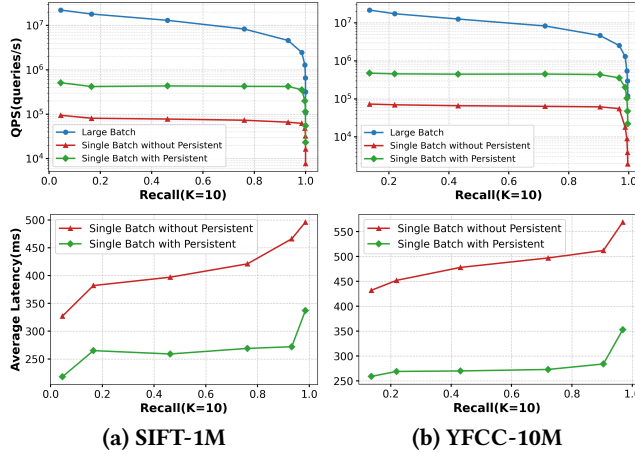


Figure 14: Persistent kernel results on SIFT and YFCC datasets.

How does the choice of specificity threshold T affect the search performance? To analyze the impact of the specificity threshold T on our algorithm, we vary T and observe its influence on performance, as shown in Fig. 15. The parameter T controls the transition between IVF-BFS-only search ($T = \infty$) and IVF-Graph-only search ($T = 0$). At $T = 0$, the graph search results in worse performance compared to BFS due to its constant overhead, as analyzed in § 4.1.2. At moderate thresholds, such as $T = 2000$ or $T = 5000$, the algorithm achieves a balanced trade-off, providing both competitive QPS and high recall. As T increases further, BFS becomes less efficient due to its exhaustive nature. This demonstrates that the threshold T is a key parameter for tuning to get the best performance.

How does VecFlow perform on different GPUs? Fig. 17 shows the comparison of VecFlow over different types of GPUs. On GH200,

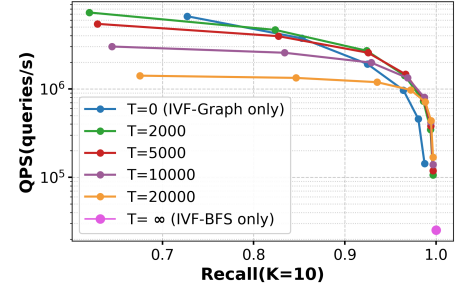


Figure 15: VecFlow varying the specificity threshold T on YFCC dataset.

VecFlow achieves 8.3M, 4M, and 300K QPS on the DEEP, YFCC, and WIKI datasets, respectively, which are 1.66x, 1.6x, and 2x faster than the results on A100. This is expected because H200 GPU has a memory bandwidth of 4.8TB/s, which is 2.4x higher than A100.

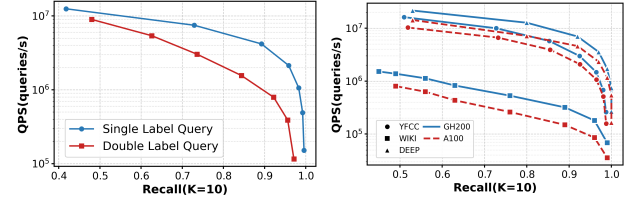


Figure 16: Performance of single label and double label VecFlow on NVidia GH200 GPU.

Does VecFlow add high index construction time? Fig. 18 compares the index construction times for CAGRA, FilteredVamana, StitchedVamana, and VecFlow. CAGRA demonstrates exceptional index build speed, primarily due to its efficient GPU-based implementation. In contrast, FilteredVamana and StitchedVamana, which rely on CPU computation, show considerably slower performance. StitchedVamana, in particular, is further hindered by the additional overhead of constructing and stitching separate indices. Our method also requires building separate indices, but we leverage multi-threading to construct multiple indices simultaneously, with each thread independently building one index. Combined with the fast index construction capabilities of CAGRA, this approach enables us to achieve significantly faster build times than both FilteredVamana and StitchedVamana, except for one case: on the YFCC-10M dataset with extensive 200K labels, building separate graph indices in IVF-Graph take longer time than FilteredVamana's single-index approach. Additionally, the time spent on the IVF-BFS component in our method is negligible since it mainly consists of memory movement and copy operations on GPU, which is highly efficient due to GPU's high memory bandwidth, avoiding the need for any extra distance computations that would be required in ANN-based graph construction processes. However, VecFlow still lags behind the single-index construction speed of CAGRA. Enhancing the efficiency of our index construction process to approach or match CAGRA's performance remains an important area of future research, as it would significantly improve the scalability and applicability of our approach in large-scale settings.

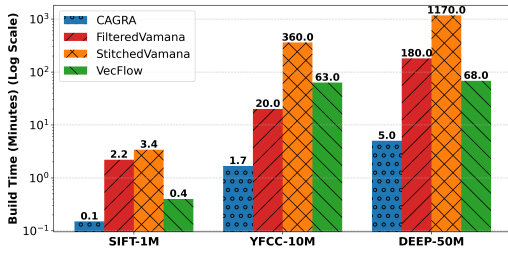


Figure 18: Index construction time comparison.

6 Conclusion

Modern AI-based applications require ANNS with filtered search. However, indexing and search algorithms with filters on GPUs must be re-designed to achieve high performance potential. We present a new GPU-based indexing and search algorithm called VecFlow, which applies a hierarchical and divide-and-conquer design of filtered-ANNS for GPUs. Furthermore, VecFlow designs redundancy-bypassing IVF-Graph and interleaved-scan based IVF-BFS to achieve high compute and memory efficiency while attaining high accuracy. Evaluation on both public and semi-synthetic datasets show that VecFlow outperforms existing solutions by two orders of magnitude in QPS and establishes the new state-of-the-art for ANNS with filters.

References

- [1] RAPIDS AI. 2025. cuVS. <https://github.com/rapidsai/cuvs>. Accessed: 2025-01-18.
- [2] Mohammad Aliannejadi, Hamed Zamani, Fabio Crestani, and W. Bruce Croft. 2018. Target Apps Selection: Towards a Unified Search Framework for Mobile Devices. In *SIGIR 2018*. 215–224.
- [3] Alexandr Andoni, Piotr Indyk, and Ilya Razenshteyn. 2018. Approximate Nearest Neighbor Search in High Dimensions. *arXiv preprint arXiv:1806.09823* (2018).
- [4] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles. In *SIGMOD 1990*. 322–331.
- [5] Ben Landrum and Magdalen Dobson Manohar and Mazin Karjekar and Laxman Dhulipala. 2024. IVF2: Fusing Classic and Spatial Inverted Indices for Fast Filtered ANNS. https://big-ann-benchmarks.com/neurips23_slides/IVF_2_filter_Ben.pdf.
- [6] Jon Louis Bentley. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM* 18, 9 (Sept. 1975), 509–517.
- [7] Philip A Bernstein, Siddharth Gollapudi, Suryansh Gupta, Ravishankar Krishnaswamy, Sepideh Mahabadi, Sandeep Silwal, Gopal R Srinivasa, Varun Suriyanarayana, Jakub Tarnawski, Haiyang Xu, et al. [n. d.]. Graph-based algorithms for nearest neighbor search with multiple filters. [n. d.].
- [8] Big-ANN. [n. d.]. NeurIPS'23 Competition Track: Big-ANN. <https://big-ann-benchmarks.com/neurips23.html>. Accessed: 2024.
- [9] Qi Chen, Haidong Wang, Mingqin Li, Gang Ren, Scarlett Li, Jeffery Zhu, Jason Li, Chuanjie Liu, Lintao Zhang, and Jingdong Wang. 2018. SPTAG: A library for fast approximate nearest neighbor search. <https://github.com/Microsoft/SPTAG>
- [10] Wei Chen, Jincai Chen, Fuhao Zou, Yuan-Fang Li, Ping Lu, Qiang Wang, and Wei Zhao. 2019. Vector and line quantization for billion-scale similarity search on GPUs. *Future Gener. Comput. Syst.* 99 (2019), 295–307.
- [11] Mostafa Dehghani, Hamed Zamani, Aliaksei Severyn, Jaap Kamps, and W. Bruce Croft. 2017. Neural Ranking Models with Weak Supervision. In *SIGIR 2017*. 65–74.
- [12] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web, WWW 2011, Hyderabad, India, March 28 - April 1, 2011*. 577–586. <https://doi.org/10.1145/1963405.1963487>
- [13] Jian Fang, Yvo TB Mulder, Jan Hidders, Jinho Lee, and H Peter Hofstee. 2020. In-memory database acceleration on FPGAs: a survey. *The VLDB Journal* 29 (2020), 33–59.
- [14] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. In *VLDB'19*.
- [15] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *CVPR 2013*.
- [16] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *VLDB'99*. 518–529.
- [17] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, et al. 2023. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.
- [18] Google. 2022. Go beyond the search box: Introducing multisearch. <https://blog.google/products/search/multisearch/>. Accessed: 2025.
- [19] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik P. A. Lensch. 2023. GGNN: Graph-Based GPU Nearest Neighbor Search. *IEEE Transactions on Big Data* 9, 1 (2023), 267–279. <https://doi.org/10.1109/TBDDATA.2022.3161156>
- [20] Jiafeng Guo, Yixing Fan, Qingyao Ai, and W. Bruce Croft. 2016. A Deep Relevance Matching Model for Ad-hoc Retrieval. In *CIKM 2016*. 55–64.
- [21] Neha Gupta. 2021. Introduction to hardware accelerator systems for artificial intelligence and machine learning. In *Advances in Computers*. Vol. 122. Elsevier, 1–21.
- [22] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *CIKM '13*. 2333–2338.
- [23] HuggingFace. 2025. WikiANN dataset. <https://huggingface.co/2024anonymous/wiki-ann>.
- [24] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in Neural Information Processing Systems* 32 (2019).
- [25] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2011. In Product Quantization for Nearest Neighbor Search. *TPAMI* 2011.
- [26] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2017. Billion-scale similarity search with GPUs. *CoRR* abs/1702.08734 (2017). [arXiv: http://arxiv.org/abs/1702.08734](http://arxiv.org/abs/1702.08734)
- [27] Yannis Kalantidis and Yannis S. Avrithis. 2014. Locally Optimized Product Quantization for Approximate Nearest Neighbor Search. In *CVPR 2014*. 2329–2336.
- [28] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and Evaluating Contextual Embedding of Source Code. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 5110–5121.
- [29] Victor Lempitsky. 2012. The Inverted Multi-index. In *CVPR '12*. 3069–3076.
- [30] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [31] Junnan Li, Dongxu Li, Caiming Xiong, and Steven C. H. Hoi. 2022. BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 12888–12900.
- [32] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data – Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2020), 1475–1488. <https://doi.org/10.1109/TKDE.2019.2909204>
- [33] Yinhan Liu, Mylène Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR* abs/1907.11692 (2019).
- [34] LongChain. [n. d.]. LongChain: Build context-aware reasoning applications. <https://github.com/langchain-ai/langchain>. Accessed: 2025.
- [35] Yury A. Malkov and D. A. Yashunin. 2016. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *CoRR* arXiv preprint abs/1603.09320 (2016).
- [36] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Distributed Representations of Words and Phrases and their Compositionality. In *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013*. 3111–3119.
- [37] Milvus-io. 2022. Milvus-docs: Conduct a hybrid search. <https://github.com/milvus-io/milvus-docs/blob/v2.1.x/site/en/userGuide/search/hybridsearch.md>. Accessed: 2025.
- [38] Bhaskar Mitra, Fernando Diaz, and Nick Craswell. 2017. Learning to Match using Local and Distributed Representations of Text for Web Search. In *WWW 2017*.
- [39] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbor Algorithms for High Dimensional Data. *TPAMI* 2014 36, 11 (2014), 2227–2240.
- [40] Mohammad Norouzi and David J. Fleet. 2013. Cartesian K-Means. In *CVPR 2013*.
- [41] Hiroyuki Ootomo, Akira Naruse, Corey J. Nolet, Ray Wang, Tamas B. Fehér, and Y. Wang. 2023. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. *2024 IEEE 40th International Conference on Data Engineering (ICDE) (2023)*, 4236–4247.
- [42] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *VLDB J.* 33, 5 (2024), 1591–1615.
- [43] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3 (2024), 120.
- [44] Inc. Pinecone Systems. 2024. Overview. <https://docs.pinecone.io/docs/overview>. Accessed: 2025.
- [45] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 8748–8763.
- [46] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.
- [47] Harshit Sharma and Anmol Sharma. 2024. A Comprehensive Overview of GPU Accelerated Databases. *arXiv preprint arXiv:2406.13831* (2024).
- [48] Harsha Simhadri. 2025. Big ANN Benchmarks. <https://github.com/harsha-simhadri/big-ann-benchmarks>. Accessed: 2025-01-18.
- [49] Harsha Vardhan Simhadri, Martin Aumüller, Amir Ingber, Matthijs Douze, George Williams, Magdalen Dobson Manohar, Dmitry Baranchuk, Edo Liberty, Frank Liu, Ben Landrum, et al. 2024. Results of the Big ANN: NeurIPS'23 competition. *arXiv preprint arXiv:2409.17424* (2024).
- [50] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *CoRR* abs/2105.09613 (2021).
- [51] Bart Thomee, David A. Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. 2016. YFCC100M: the new data in multimedia research. *Commun. ACM* 59, 2 (2016), 64–73.
- [52] Karthik V., Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedula. 2024. BANG: Billion-Scale Approximate Nearest Neighbor Search using a Single GPU. *arXiv: [cs.DC]* <https://arxiv.org/abs/2401.11324>

- [53] Christophe Van Gysel, Maarten de Rijke, and Evangelos Kanoulas. 2016. Learning Latent Vector Spaces for Product Search. In *CIKM '16*. 165–174.
- [54] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongkang Ni. 2023. An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- [55] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *arXiv preprint arXiv:2101.12631* (2021).
- [56] Weaviate. 2022. Weaviate Documentation: Filters. <https://weaviate.io/developers/weaviate/current/graphql-references/filters.html>. Accessed: 2025.
- [57] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: a hybrid analytical engine towards query fusion for structured and unstructured data. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 3152–3165. <https://doi.org/10.14778/3415478.3415541>
- [58] Lei Yu, Karl Moritz Hermann, Phil Blunsom, and Stephen Pulman. 2014. Deep Learning for Answer Sentence Selection. *CoRR* abs/1412.1632 (2014).
- [59] Yuanhang Yu, Dong Wen, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2022. GPU-accelerated Proximity Graph Approximate Nearest Neighbor Search and Construction. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 552–564.
- [60] Hamed Zamani, Bhaskar Mitra, Xia Song, Nick Craswell, and Saurabh Tiwary. 2018. Neural Ranking Models with Multiple Document Fields. In *WSDM '18*.
- [61] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 377–395. <https://www.usenix.org/conference/osdi23/presentation/zhang-qianxi>
- [62] Zili Zhang, Fangyue Liu, Gang Huang, Xuanzhe Liu, and Xin Jin. 2024. Fast Vector Query Processing for Large Datasets Beyond {GPU} Memory with Re-ordered Pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 23–40.
- [63] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*. IEEE, 1033–1044.